

Frontend Interview Study Pack

Sumit Kaktwan | 3 YOY | Target: 12+ LPA

JS | React | Next.js | TypeScript | CSS | DSA

DOCUMENT 1: JavaScript - Basic

Q: What are JavaScript data types?

A: 7 Primitives: string, number, boolean, undefined, null, bigint, symbol.

1 Non-primitive

Q: Difference between var, let, and const?

A: var: function-scoped, hoisted as undefined, can redeclare.

let: block-scoped

Q: What is hoisting?

A: JS moves declarations to top during compilation.

Function declarations

Q: Difference between == and ===?

A: == performs type coercion ('5' == 5 is true).

=== is strict equality

Q: Difference between null and undefined?

A: undefined: variable declared but not assigned.

null: intentional empty value

Q: Explain map(), filter(), reduce()?

A: map(): transforms each element, returns new array.

filter(): keeps elements that pass a test

Q: Difference between slice() and splice()?

A: slice(start, end): returns new array, does NOT mutate.

splice(start, deleteCount, ...items)

Q: Arrow functions vs regular functions?

A: Arrow: lexical this, no arguments object, cannot use new.

Regular: function keyword, curly braces, return statement

Q: What is destructuring?

A: Extract values from arrays/objects into variables.

const [a, b] = [1, 2];

Q: Spread vs Rest operator?

A: Spread (...): expands iterable. [...arr1, ...arr2], {...obj}.

Rest (...): collects remaining arguments

Q: Nullish coalescing (??) vs OR (||)?

A: ?? returns right side only for null/undefined.

|| returns right side if left is falsy

Q: for...in vs for...of?

A: for...in: iterates object keys.

for...of: ite

Q: Important string methods?

A: split(), join(), slice(), includes(), toUpperCase(), toLowerCase(),

trim(), rep

Q: Important array methods?

A: push/pop (end), unshift/shift (start), find, findIndex, some, every,

includes, s

Q: How to sort numbers correctly?

A: arr.sort((a,b) => a-b) for ascending.

Without co

DOCUMENT 2: JavaScript - Intermediate

Q: What is a closure?

A: Function bundled with its lexical environment. Inner function accesses outer variables even after outer returns.

Classic: u

Q: Explain the this keyword?

A: Global: window. Object method: the object. Arrow: lexical (outer scope).

call/apply/

Q: call() vs apply() vs bind()?

A: call(obj, arg1, arg2): invoke immediately, args individually.

apply(obj,

Q: What is currying?

A: Transforming multi-arg function into sequence of single-arg functions.

const add

Q: Callback vs Promise vs async/await?

A: Callback: nested, callback hell.

Promise: .

Q: Promise states and APIs?

A: States: pending, fulfilled, rejected.

Promise.a

Q: Explain async/await?

A: async function always returns Promise.

await pau

Q: Explain the Event Loop?

A: JS is single-threaded. Order: Sync code -> ALL microtasks (Promise.then) -> ONE macrotask (setTimeout) -> repeat.

Example:

Q: Debounce vs Throttle?

A: Debounce: waits until user stops triggering (search input).

Throttle: e

Q: Shallow copy vs Deep copy?

A: Shallow: {...obj}, [...arr], Object.assign - nested refs shared.

Deep: stru

Q: What is IIFE?

A: Immediately Invoked Function Expression. Runs immediately, creates private scope.

(function()

Q: Higher-order function?

A: Takes function as argument OR returns function.

map, filter

Q: What is a pure function?

A: Same input always gives same output. No side effects.

DOCUMENT 3: JavaScript - Advanced

Q: Prototypal inheritance?

A: Every object has `[[Prototype]]` link. Properties not found on object are looked up the chain.

const dog

Q: Ways to create objects?

A: Object literal, `new Object()`, `Object.create()`, Constructor+`new`, ES6 class, Factory function, `JSON.parse()`.

Q: Event bubbling, capturing, delegation?

A: Phases: Capturing -> Target -> Bubbling.

stopPropa

Q: ES6 Modules?

A: `export const fn = ...`; `export default Component`;

import De

Q: Map vs Object?

A: Map: any key type, `.size` property, directly iterable, better for frequent add/delete.

Object: str

Q: Generator functions?

A: `function* gen() { yield 1; yield 2; }`

Can paus

Q: Polyfill for `Array.map`?

A: `Array.prototype.myMap = function(cb) { const r=[]; for(let i=0;i<this.length;i++) r.push(cb(this[i],i,this)); return r; };`

Q: Polyfill for `Function.bind`?

A: `Function.prototype.myBind = function(ctx,...args) { const fn=this; return (...a) => fn.apply(ctx,[...args,...a]); };`

Q: Implement `once(fn)`?

A: `function once(fn) { let called=false, result; return (...args) => { if(!called){called=true;result=fn(...args);} return result; }; }`

Q: Implement `memoize(fn)`?

A: `function memoize(fn) { const cache=new Map(); return (...args) => { const k=JSON.stringify(args); if(cache.has(k))return cache.get(k); const r=fn(...args); cache.set(k,r); return r; }; }`

DOCUMENT 4: HTML - Basic to Advanced

Q: What is <!DOCTYPE html>?

A: Tells browser to render in standards mode, not quirks mode. First line of HTML5.

Q: Semantic HTML examples?

A: header, nav, main, article, section, aside, footer, figure, time.

Benefits: S

Q: div vs span?

A: div: block-level container. span: inline text wrapper.

Q: async vs defer in script tags?

A: async: parallel download, executes immediately, order not guaranteed.

defer: par

Q: localStorage vs sessionStorage vs cookies?

A: localStorage: ~5MB, never expires, not sent to server.

sessionSt

Q: What is CORS?

A: Browser blocks cross-origin requests unless server sends Access-Control-Allow-Origin header.

Q: SEO meta tags?

A: description, viewport, og:title, og:image, robots index/follow.

Q: Accessibility (a11y)?

A: alt text on images, label for inputs, aria-label, semantic HTML, keyboard navigation.

Q: data-* attributes?

A: Custom HTML attributes. Access via element.dataset.userId in JS.

DOCUMENT 5: CSS - Basic to Advanced

Q: CSS Box Model?

A: margin > border > padding > content.

box-sizing

Q: CSS Specificity?

A: Element(1) < Class(10) < ID(100) < Inline < !important

Q: display: block vs inline vs inline-block?

A: block: new line, full width. inline: no new line, no width/height.

inline-block

Q: Flexbox?

A: 1D layout. display:flex; justify-content:center; align-items:center;

Best way

Q: CSS Grid?

A: 2D layout. grid-template-columns: repeat(3, 1fr);

Use for pa

Q: Pseudo-classes vs pseudo-elements?

A: Pseudo-class (state): :hover, :focus, :nth-child.

Pseudo-el

Q: Responsive design?

A: Mobile-first: base styles + @media (min-width: 768px).

viewport m

Q: CSS position values?

A: static, relative, absolute, fixed, sticky.

Q: BEM naming?

A: Block: .card, Element: .card__title, Modifier: .card--featured.

Q: Tailwind CSS?

A: Utility-first CSS. Compiles only used classes via PostCSS purge.

DOCUMENT 6: React - Basic

Q: What is React?

A: JS library for building UIs. Component-based, Virtual DOM, one-way data flow.

Disadvant

Q: What is JSX?

A: HTML-like syntax compiling to `React.createElement()`.

classNam

Q: Props vs State?

A: Props: from parent, read-only, configure component.

State: inte

Q: Virtual DOM (full answer)?

A: Lightweight JS object tree with element type, props, children.

NOT a cop

Q: Why key in lists?

A: Helps React identify changed items. Use stable unique IDs, never index if list reorders.

Q: Conditional rendering?

A: Ternary: `{cond ? <A/> : <B/>}`. AND: `{error && <Error/>}`.

Q: useState?

A: `const [val, setVal] = useState(init)`.

Never mu

Q: useEffect - 3 cases?

A: No deps: every render. `[]`: once on mount. `[dep]`: mount + when dep changes.

Cannot be

DOCUMENT 7: React - Intermediate

Q: Prop drilling and solutions?

A: Passing props through many layers. Fix: Context API, Redux, composition, custom hooks.

Q: Context API?

A: createContext -> Provider wraps tree -> useContext in any child.

Good for t

Q: Controlled vs Uncontrolled?

A: Controlled: React state is source (value + onChange).

Uncontrol

Q: useRef?

A: Mutable reference persisting across renders without re-render.

Use for D

Q: useMemo vs useCallback?

A: useMemo: memoizes computed VALUE. useCallback: memoizes FUNCTION reference.

Use with F

Q: React.memo?

A: Prevents re-render if props unchanged (shallow compare).

Q: useReducer?

A: Complex state logic. dispatch({type:'INCREMENT'}). Like Redux locally.

Q: React Router?

A: BrowserRouter > Routes > Route path element.

useParam

Q: Custom hooks?

A: Reusable functions using other hooks. useFetch, useDebounce, useLocalStorage.

Q: HOC?

A: Function taking component, returning enhanced component. Modern alt: custom hooks.

Q: React.lazy + Suspense?

A: Code splitting. lazy(() => import('./Comp')). Wrap in Suspense with fallback.

Q: Error Boundaries?

A: Class component catching child errors. getDerivedStateFromError + componentDidCatch.

Does NOT

DOCUMENT 8: React - Advanced

Q: Reconciliation / Diffing?

A: React compares old vs new VDOM, calculates minimum DOM changes.

Different t

Q: What causes re-renders?

A: State change, props change, context change, parent re-render.

Prevent: n

Q: useEffect vs useLayoutEffect?

A: useEffect: after paint (API calls). useLayoutEffect: before paint (DOM measurements).

Q: SSR vs CSR?

A: CSR: browser renders, poor SEO, slow first paint.

SSR: serv

Q: Performance optimization?

A: Code splitting, memoization, virtualization, debounce, avoid inline fns,

image opt

Q: Class lifecycle to Hooks?

A: componentDidMount=useEffect([], []). componentDidUpdate=useEffect([dep]).

willUnmou

Q: React Fiber?

A: Reimplementation enabling interruptible rendering and concurrent features.

Q: JWT Auth flow?

A: Login -> token -> store -> attach to headers -> 401 -> redirect/refresh.

Q: Optimistic updates?

A: Update UI immediately, rollback if API fails.

DOCUMENT 9: Next.js - Basic to Advanced

Q: Why Next.js over CRA?

A: File routing, SSR/SSG, API routes, auto code splitting, next/image optimization.

Q: SSR vs SSG vs CSR vs ISR?

A: CSR: browser, poor SEO, dashboards.

SSR: serv

Q: File-based routing?

A: pages/index.js = /. pages/blog/[slug].js = /blog/my-post.

app/page.

Q: App Router vs Pages Router?

A: App: app/ dir, Server Components default, layout.tsx.

Pages: pa

Q: Server vs Client Components?

A: Server (default): no hooks, async data fetching.

Client ('us

Q: next/image?

A: Auto resize, WebP, lazy load. priority prop for above-fold images.

Q: API Routes?

A: app/api/users/route.js: export GET, POST functions.

Q: Why SEO friendly?

A: Server sends complete HTML. Google crawler sees content without waiting for JS.

DOCUMENT 10: TypeScript - Basic to Advanced

Q: Why TypeScript?

A: Compile-time error catching, better IDE support, self-documenting, easier refactoring.

Q: Basic types?

A: string, number, boolean, any (avoid), union (string|number), arrays (string[]).

Q: Interface vs Type?

A: Interface: object shapes, extends. Type: unions, intersections.

type Admin

Q: Generics?

A: function wrap<T>(value: T): T[] { return [value]; }

interface A

Q: Utility types?

A: Partial<T>, Required<T>, Readonly<T>, Pick<T,K>, Omit<T,K>, Record<K,V>.

Q: React + TypeScript?

A: interface Props { label: string; onClick: () => void; }

useState<

Q: How TS runs?

A: TS -> tsc (type check) -> JS -> Browser.

DOCUMENT 11: Redux & State Management

Q: When to use Redux?

A: Shared state across many components, complex logic, middleware needed.

Don't use

Q: Redux flow?

A: UI dispatches action -> reducer updates state -> store notifies -> UI re-renders.

Q: Redux Toolkit?

A: createSlice with name, initialState, reducers.

useSelect

Q: Redux vs Context?

A: Context: simple, all consumers re-render. Redux: selective, DevTools, middleware.

DOCUMENT 12: Testing (Jest + RTL)

Q: Why test?

A: Catch bugs early, refactoring confidence, documentation.

Q: Jest + RTL basics?

A: `render(<Component/>). screen.getByRole('button'). userEvent.click()`.

`expect(sc`

Q: Query priority?

A: `getByRole` > `getByLabelText` > `getByPlaceholderText` > `getByText` > `getByTestId`.

Q: Test async?

A: `await waitFor(() => expect(screen.getByText('Sumit')).toBeInTheDocument())`.

Q: What to test?

A: Renders, interactions, loading/error states. NOT implementation details.

DOCUMENT 13: DSA for Frontend

Q: DSA patterns for frontend?

A: Find pair: Hash Map (Two Sum). Substring: Sliding Window.

Sorted arr

Q: Two Sum solution?

A: `const map=new Map(); for(let i=0;i<nums.length;i++){const c=target-nums[i]; if(map.has(c))return[map.get(c),i]; map.set(nums[i],i);}`

Time O(n)

Q: Remove duplicates?

A: [...new Set(arr)] or filter with indexOf.

Q: Valid Palindrome?

A: Clean string, compare with reversed. Or two pointers.

Q: Longest substring without repeating?

A: Sliding window with Set. O(n) time.

Q: Flatten array?

A: `arr.reduce((a,i) => a.concat(Array.isArray(i)?flatten(i):i), []).`

Q: Valid Anagram?

A: Count characters in both strings with hash map.

Q: Maximum Subarray (Kadane)?

A: `curr = max(nums[i], curr+nums[i]); max = max(max, curr).` O(n).

Q: Binary Search?

A: `let l=0,r=arr.length-1; while(l<=r){mid=Math.floor((l+r)/2); ...}` O(log n).

Q: 30 problems to practice?

A: Easy: Reverse String, Palindrome, Two Sum, Anagram, FizzBuzz, Binary Search.

Medium: L

DOCUMENT 14: Machine Coding

Q: Machine coding rules?

A: Ask clarifying questions. Plan structure. UI first then logic. Mention edge cases.

Q: Todo App?

A: Add, toggle, delete, filter ALL/ACTIVE/COMPLETED. useState array.

Q: Timer?

A: Start/stop/reset, continues from pause. useRef for interval, useEffect cleanup.

Q: Debounced Search?

A: API data, debounce 300ms, reset button, delete row. useCallback + debounce.

Q: Star Rating?

A: Hover state, click to set, gold/gray stars.

Q: Modal?

A: Fixed overlay, click outside, ESC key, stopPropagation.

Q: Infinite Scroll?

A: IntersectionObserver on last item, page state, loading flag.

Q: Multi-step Form?

A: Step state, shared formData, sessionStorage for refresh recovery.

Q: Pagination?

A: Math.ceil(total/perPage), slice data, disable prev/next at bounds.

Q: Accordion?

A: Set for open items, single or multiple mode.

Q: Theme Toggle?

A: Context + localStorage + CSS variables on documentElement.

Q: C-Shape Box Click?

A: Track click order, all green then reset one-by-one with 1s delay.

Q: Quiz App?

A: API questions, next/prev, progress circles green/red.

DOCUMENT 15: Frontend System Design

Q: System design framework?

A: Requirements -> Component architecture -> State management -> Data fetching ->

Performan

Q: Design Netflix homepage?

A: Header, HeroBanner, ContentRow[] with MovieCard.

React Que

Q: Design Autocomplete?

A: Debounce 300ms, min 2 chars, AbortController, keyboard nav, cache recent.

Q: Design Infinite Scroll?

A: IntersectionObserver, cursor pagination, virtualize 1000+ items.

Q: Handle 1 million users?

A: CDN, code splitting, image optimization, React Query cache,

virtualizati

Q: Web Vitals targets?

A: LCP < 2.5s, FID < 100ms, CLS < 0.1.

DOCUMENT 16: Git, HTTP & Browser

Q: Essential Git commands?

A: clone, checkout -b, add, commit, push, pull, merge, stash, log, diff, reset --soft.

Q: HTTP status codes?

A: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden,

404 Not F

Q: HTTP methods?

A: GET (read), POST (create), PUT (replace), PATCH (partial), DELETE (remove).

Q: Critical Rendering Path?

A: HTML->DOM, CSS->CSSOM, Render Tree, Layout, Paint, Composite.

Q: Browser storage?

A: localStorage (5MB, permanent), sessionStorage (tab), Cookie (4KB, sent to server).

DOCUMENT 17: Behavioral & HR

Q: Tell me about yourself?

A: Sumit, 3 YOE frontend at Trifler. React, Next.js, TypeScript, Tailwind.

Built onbo

Q: STAR format?

A: Situation, Task, Action, Result. Use for behavioral questions.

Q: Why leave current company?

A: Grown at Trifler, seeking product company with scale and growth.

Q: Expected CTC?

A: 12-14 LPA based on 3 YOE and Bangalore market. Open to discussion.

Q: 5 stories to prepare?

A: Challenging project, bug fixed, mistake+learning, conflict, initiative.

Q: Questions for interviewer?

A: Team structure, tech stack, first 3 months, biggest challenge, code reviews.

DOCUMENT 18: Coding Implementations Cheat Sheet

Q: Debounce implementation?

A: `function debounce(fn,d){let t;return(...a)=>{clearTimeout(t);t=setTimeout(()=>fn(...a),d);;}}`

Q: Throttle implementation?

A: `function throttle(fn,l){let t=false;return(...a)=>{if(!t){fn(...a);t=true;setTimeout(()=>t=false,l);}};}`

Q: once implementation?

A: `function once(fn){let c=false,r;return(...a)=>{if(!c){c=true;r=fn(...a);}return r;}}`

Q: useDebounce hook?

A: useState debounced value. useEffect with setTimeout and cleanup on value change.

Q: useFetch hook?

A: useState data/loading/error. useEffect fetch on url change. Return {data,loading,error}.

Q: useLocalStorage hook?

A: Initialize from localStorage. useEffect sync on change.

Q: Promise.all polyfill?

A: Track completed count. Resolve when all done. Reject on first failure.

Q: DSA time complexity?

A: Two Sum $O(n)$, Binary Search $O(\log n)$, Longest Substring $O(n)$, Max Subarray $O(n)$.

APPENDIX: 6-Week Study Plan

Q: 6-Week Study Plan?

A: Week 1: JS Basic+Intermediate+DSA easy (Doc 1-3, 13).

Week 2: F

Q: Daily routine?

A: Hour 1: Theory (5 Q&A). Hour 2: 2 DSA problems. Hour 3: 1 machine coding.

Hour 4: R

Q: Gap fix priority?

A: 1. Event Loop 2. Virtual DOM deep answer 3. DSA 40+ problems

4. Machin

Q: Target companies for 12 LPA?

A: Product startups, SaaS, fintech in Bangalore. Range: 10-16 LPA for 3 YOE.